

# Interprocess Communication

Jibon Naher<sup>1</sup> and Samnun Azfar<sup>2</sup>

<sup>1</sup>Lecturer, CSE

<sup>2</sup>Junior Lecturer, CSE

February 26, 2026

## 1 Race Condition

### 1.1 What Is a Race Condition?

A race condition occurs when two or more processes (or threads) access shared data concurrently, and the final result depends on the order of execution. Because the order of execution is unpredictable, the program can produce inconsistent or incorrect results.

### 1.2 Critical Section

The part of code accessing shared data — must be executed by only one process at a time. for example, line 8 of above code: *counter++*;

### 1.3 Example 1: Race Condition in C (Two Threads Incrementing a Variable)

```
1  #include <stdio.h>
2  #include <pthread.h>
3
4  int counter = 0; // Shared resource
5
6  void* increment(void* arg) {
7      for (int i = 0; i < 100000; i++) {
8          counter++; // Critical section
9      }
10     return NULL;
11 }
12 int main() {
13     pthread_t t1, t2;
14     pthread_create(&t1, NULL, increment, NULL);
15     pthread_create(&t2, NULL, increment, NULL);
16
17     pthread_join(t1, NULL);
18     pthread_join(t2, NULL);
19
20     printf("Final counter value: %d\n", counter);
21     return 0;
22 }
```

## 1.4 Explanation

The following section explains the function of key lines typically found in C programs that use the `pthread` library for thread management.

### 1.4.1 Line 13: `pthread_t t1, t2;`

Declares two thread identifiers (`t1` and `t2`). These are like "thread handles," used by the main program to refer to the threads later (e.g., to wait for them to finish).

### 1.4.2 Line 14: `pthread_create(&t1, NULL, increment, NULL);`

This is the function call to **create a new thread** that runs the `increment` function concurrently.

- `&t1`: Address of the thread ID variable where the newly created thread's identifier will be stored.
- `NULL`: Specifies the thread attributes (e.g., stack size, priority). `NULL` means default attributes are used.
- `increment`: The function pointer that the new thread will begin execution in.
- `NULL`: The single argument to be passed to the `increment` function. (In this case, none is passed.)

**Note:** After this line, two threads are running concurrently: the main thread (continuing in `main`) and Thread `t1` (running `increment`).

### 1.4.3 Line 15: `pthread_create(&t2, NULL, increment, NULL);`

Similar to the above, this creates a second thread, `t2`. Now, **three threads exist** (`main`, `t1`, and `t2`). Threads `t1` and `t2` are now competing to modify the shared resource, leading to the race condition.

### 1.4.4 Line 17: `pthread_join(t1, NULL);`

This is a blocking call. The main thread will pause and wait for **thread** `t1` to finish its execution (i.e., exit the `increment` function). This is crucial for ensuring the main thread does not continue until the concurrent work is complete.

### 1.4.5 Line 18: `pthread_join(t2, NULL);`

The main thread waits for **thread** `t2` to finish. After both `pthread_join()` calls successfully return, both worker threads are guaranteed to be finished, and it is safe for the main thread to proceed, typically by reading the final result of the shared counter.

## 1.5 Execute the program

command: `gcc race.c -o race -pthread` and then `./race`

## 2 How to fix: using mutex

```
1  #include <pthread.h>
2  pthread_mutex_t lock = PTHREAD_MUTEX_INITIALIZER;
3  int counter = 0;
4
5  void* increment(void* arg) {
6      for (int i = 0; i < 100000; i++) {
7          pthread_mutex_lock(&lock);
8          counter++;
9          pthread_mutex_unlock(&lock);
10     }
11     return NULL;
12 }
```

## 3 Another Fix: Using Semaphore

```
1  #include <stdio.h>
2  #include <pthread.h>
3  #include <semaphore.h> // Required for semaphores
4
5  int counter = 0; // Shared resource
6
7  // Declare a semaphore (initialized later)
8  sem_t sem;
9
10 void *increment(void *) {
11     for (int i = 0; i < 100000; i++) {
12         sem_wait(&sem); // Acquire the semaphore (enter critical
13                          // section)
14         counter++; // Critical section (now safe!)
15         sem_post(&sem); // Release the semaphore (exit critical
16                          // section)
17     }
18 }
19
20 int main() {
21     pthread_t t1, t2;
22
23     // Initialize semaphore to 1 (binary semaphore = mutex behavior)
24     sem_init(&sem, 0, 1);
25
26     pthread_create(&t1, NULL, increment, NULL);
27     pthread_create(&t2, NULL, increment, NULL);
28
29     pthread_join(t1, NULL);
30     pthread_join(t2, NULL);
31
32     // Destroy the semaphore
33     sem_destroy(&sem);
34
35     printf("Final counter value: %d\n", counter);
36     return 0;
37 }
```